

From Language to Action: Can LLM-Based Agents Be Used for Embodied Robot Cognition?

Shinas Shaji^{1,2}, Fabian Huppertz¹, Alex Mitrevski³, Sebastian Houben^{1,2}
¹Hochschule Bonn-Rhein-Sieg ²Fraunhofer IAIS ³Chalmers University of Technology

Introduction

Motivation

- Robots in everyday environments need **task planning, execution monitoring, and failure recovery**
- Traditional cognitive architectures require extensive engineered knowledge and many dedicated components
- LLMs exhibit emergent capabilities in reasoning, language understanding, and task execution

Research Question

Can an LLM (used *zero-shot*) serve as the cognitive core of an embodied robot in a 3D household environment?

Contributions

- Cognitive architecture** embedding an agentic LLM with episodic memory, working memory, and a tool-calling interface for navigation and manipulation
- Empirical evaluation** on two household tasks across four LLMs (GPT-4.1, Claude 4 Sonnet, Qwen3 Coder 480B, DeepSeek V3.1)

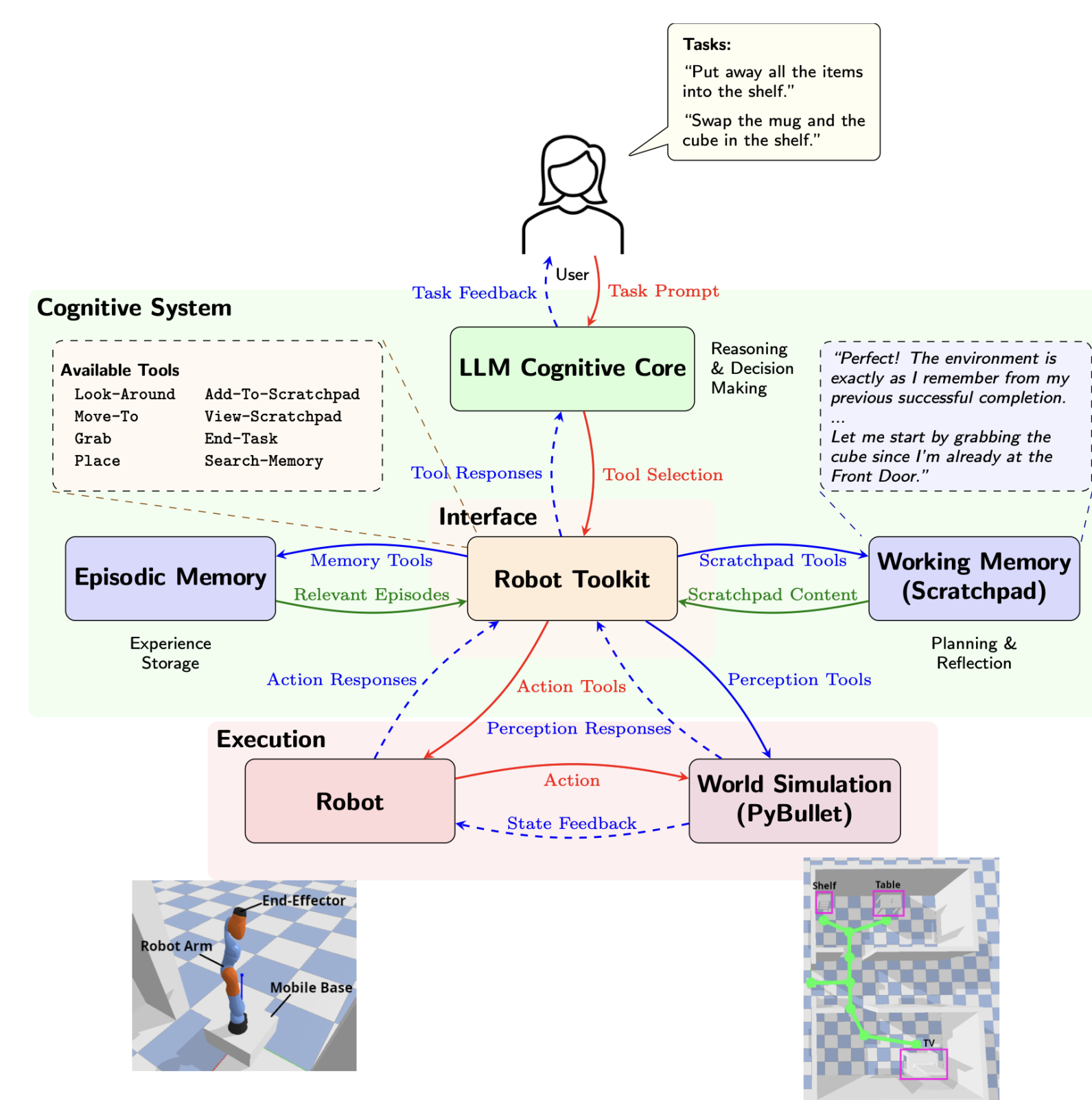


Fig. 1: System overview

System Architecture

The robot is embodied as a **mobile manipulator** (omnidirectional base + 7-DOF arm) operating in a PyBullet-simulated household environment with a kitchen, living room, and hallway.

Memory Architecture

- Working memory:** LLM context + scratchpad for immediate task reasoning and planning
- Episodic memory:** ChromaDB vector store of past execution summaries, semantically searchable for relevant prior experiences

Tools (callable actions for the LLM)

- Look-Around — perceive nearby objects and current location

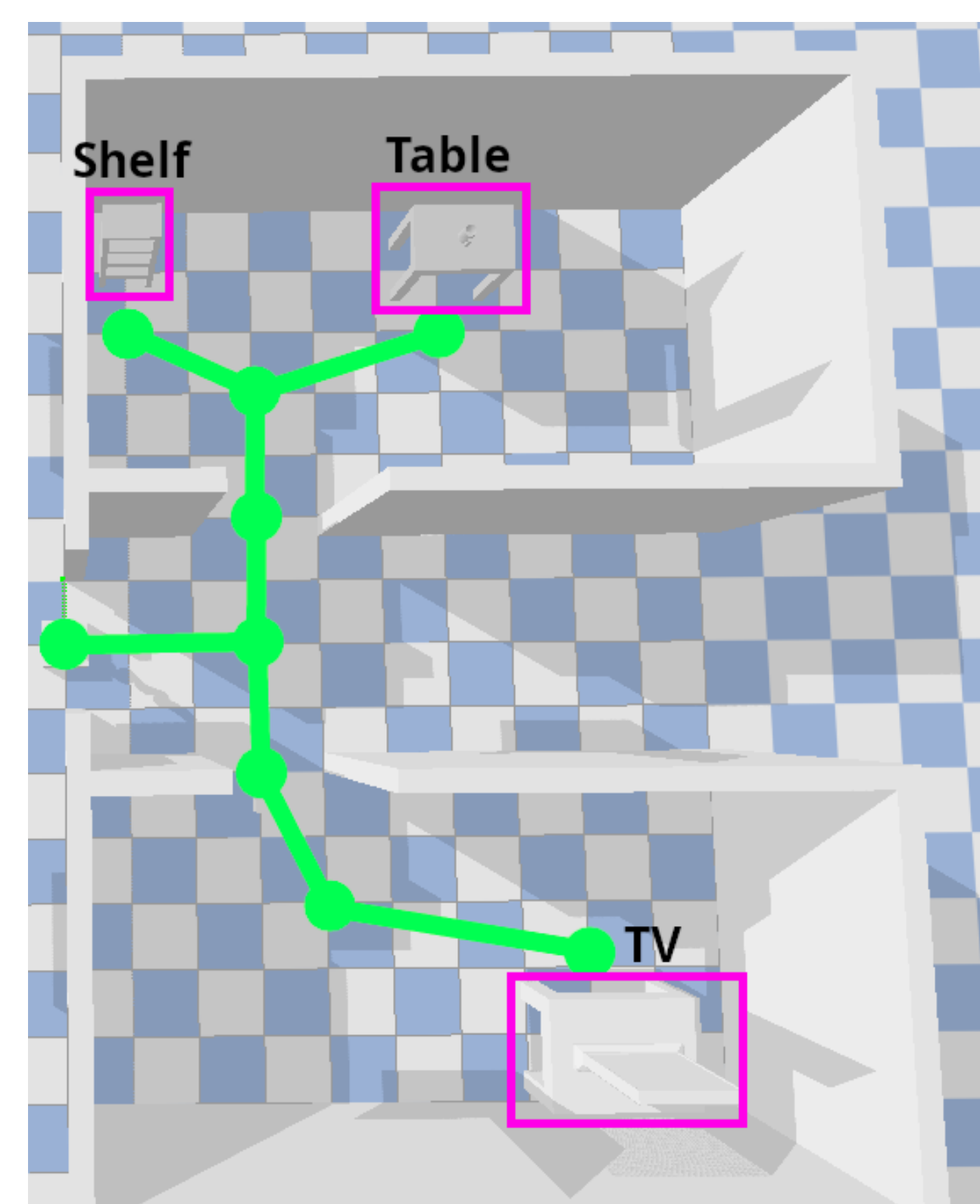


semantic map location via A*

- Move-To — move end-effector (if within reach)

- Place — release held object at a target location

- Add-To-Scratchpad / View-Scratchpad — plan and reason



Evaluation

Tasks (static setup, 3D simulated household)

- T1 – Placing items:** place a mug, box, and cube into a three-shelf cupboard
- T2 – Swapping items:** swap mug and cube (requires reasoning about a temporary storage location; executed after T1)

Execution Success Rate

	GPT-4.1	Claude 4	Qwen3 480B	DeepSeek V3.1
T1	100%	100%	80%	100%
T2	44.4%	100%	66.2%	75.5%

All models reliably complete T1; T2 is more challenging (requires multi-step reasoning about a temporary swap location). **Claude 4 Sonnet achieves 100% on both tasks**, but with high latency (30–150 s/request).

Hallucinated Success

All models (except Claude) *overestimate* their success — they believe they have succeeded even when the world state is incorrect. DeepSeek V3.1 is persistently overconfident, claiming success in every trial regardless of outcome.

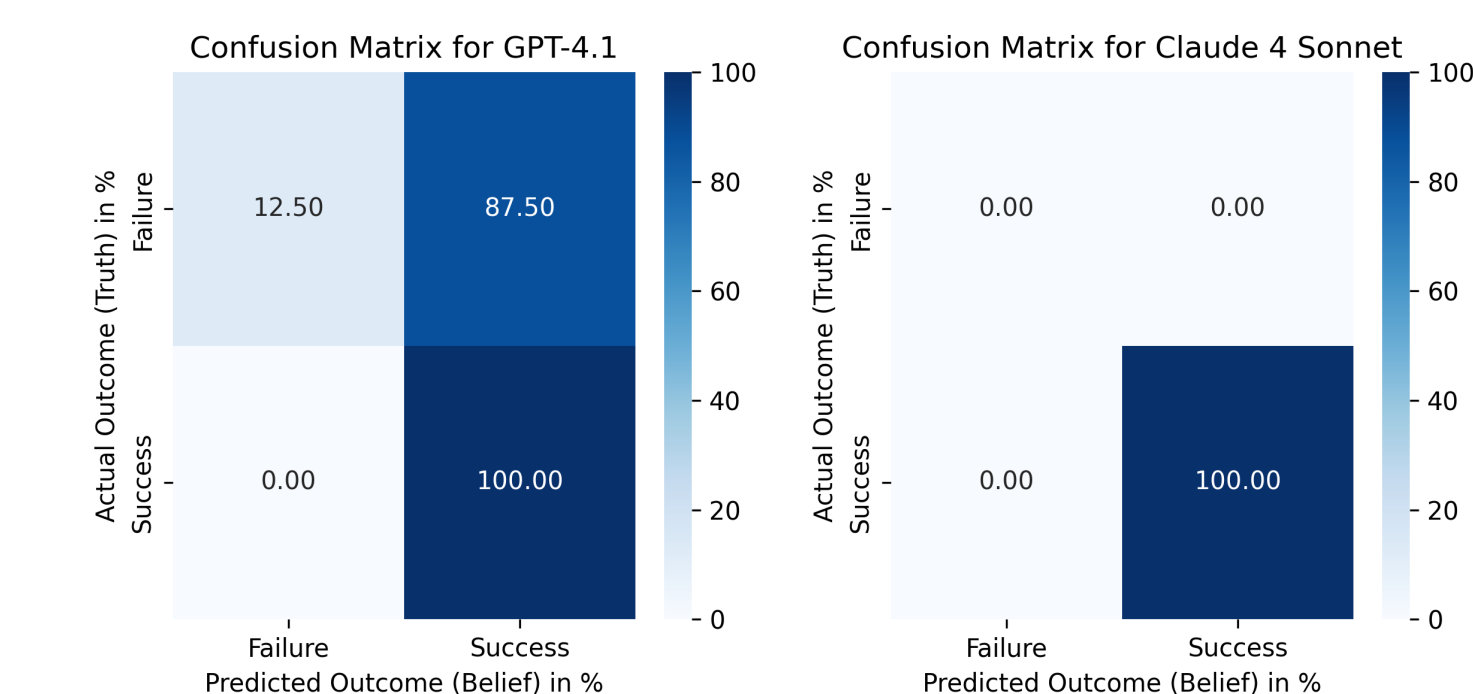


Fig. 2: Believed vs. actual success: GPT-4.1 (left) and Claude 4 Sonnet (right)

Effect of Episodic Memory

- Memory *reduces tool calls* on average for all models and tasks
- Success rate improvement is modest and model-dependent; hallucinated memory labels limit benefits

Discussion & Conclusions

Key Findings

- LLMs can reason about spatial layouts, sequence tool calls, and **recover from failures without explicit replanning logic**
- Episodic memory provides modest improvements; benefits are limited by **hallucinated success labels** corrupting the memory store
- Models misinterpret ambiguous instructions (e.g., attempting to grab a TV when told to “put all objects away”)
- Smaller models (~24B parameters) cannot coherently chain actions; large frontier models are necessary

Limitations

- Overconfidence:** agents incorrectly believe they succeeded, impairing episodic memory quality
- Task refusal:** after completing one task, models sometimes hallucinate that a subsequent task is already done
- High latency** of cloud-hosted models (30–150 s/request) limits real-robot activity
- Evaluation is simulation-only; real-robot deployment not yet validated

Future Work